

Architecture and System Design

Date	Sept 10, 2024
Author	@ Rajeesh CV
Reviewed By	
Status	Approved

1. Introduction and Goals

This document describes the architecture of **Emotist**, a digital mental wellness platform, following the [arc42](#) architecture template. It is intended for developers, architects, stakeholders, and new team members who need to understand the system’s structure, core decisions, and design rationale. The goal is to ensure consistent development practices, ease onboarding, and support maintainability and scalability of the platform.

1.1. Requirements Overview

Emotist is a comprehensive platform designed to facilitate **online therapy sessions** and **mental wellness programs**. It connects **therapists** and **clients** through a secure, scalable, and user-friendly interface. Key features include:

- Therapist onboarding and profile management
- Client onboarding via invitations or self-signup
- Booking and managing therapy sessions
- Real-time communication via integrated video calls
- Messaging between therapists and clients
- Task and note management for therapy sessions

The platform is built using a **modular, Domain-Driven Design (DDD)** approach and adheres to **Clean Architecture** principles. It supports **event-driven communication** between bounded contexts and follows **CQRS** for scalability and separation of concerns.

1.2. Quality Goals

The primary quality goals of the Emotist platform are:

- **Maintainability:** Clean layering and modular structure allow teams to work independently on different features and bounded contexts.
- **Scalability:** Event-driven and asynchronous communication supports scaling specific parts of the system independently.
- **Extensibility:** Easily add new features or integrate with external systems.
- **Reliability:** Ensure consistent and error-tolerant behaviour.

1.3. Stakeholders

Name	Role	Contact	Expectation
Rajeesh	Architect	@ Rajeesh CV	Scalable and extensible architecture
Siby	Engineering Manager	@ Siby Mathew	Timely delivery with higher quality
Sajir	Dev Group Lead	@ Sajir T	• Clean and modular code, with 80% test coverage • Highly observable application

2. Architecture Constraints

A combination of technical, organisational, and regulatory constraints shapes the architecture of the **Emotist** platform. These constraints are non-negotiable factors that influence decisions regarding system structure, technology choices, and implementation strategy.

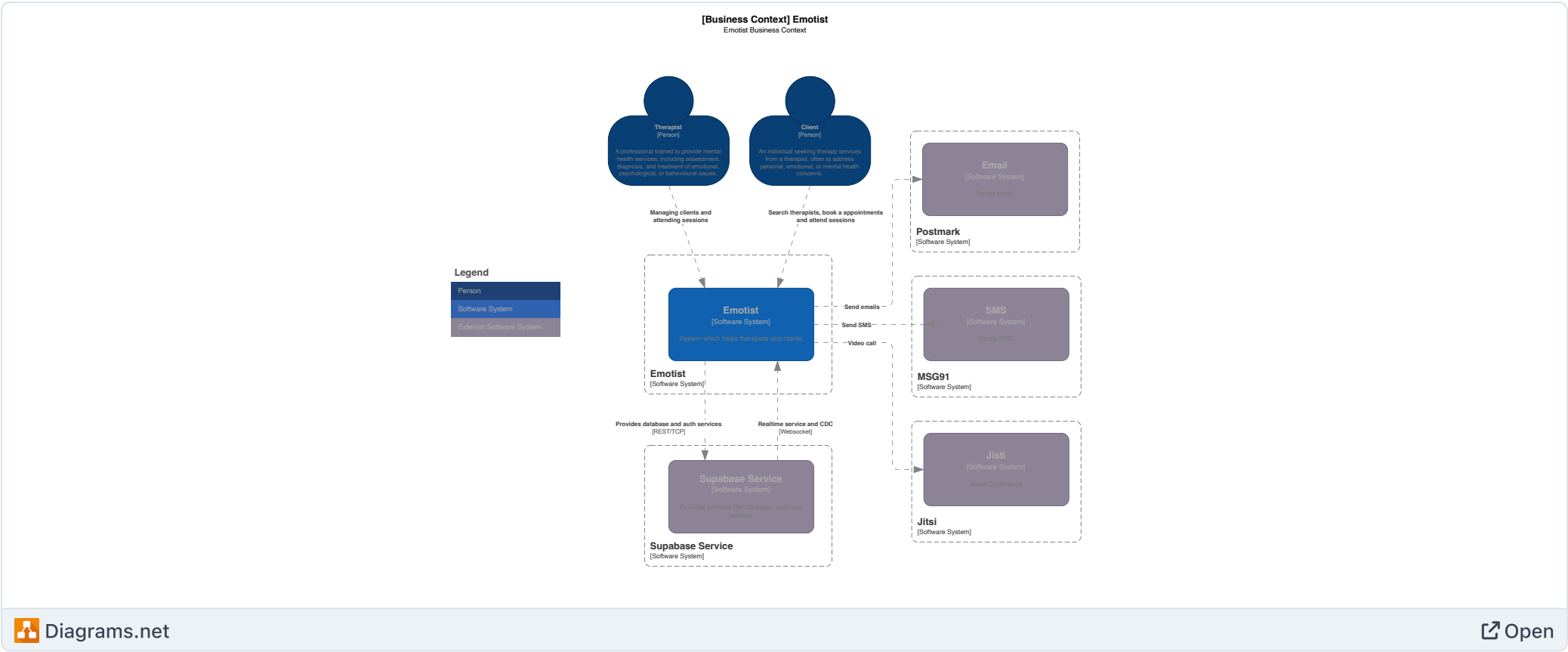
2.1. Technical Constraints

Constraints	Description
Frontend	Frontend must React with TypeScript and use React i18n for localisation
Backend	Backend must use NestJS with TypeScript, and follow DDD with clean architecture.
Database	Use PostgreSQL for all persistent storage, with use of Row-Level Security (RLS) .

Message Queue	Use BullMQ (Redis-based queues) for event publishing and cross-bounded context communication.
Observability	Must support OpenTelemetry for tracing, metrics, and logging.

3. System Scope and Context

3.1. Business Context

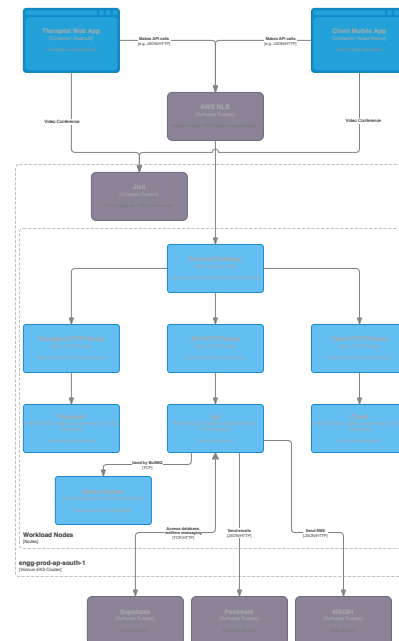


Name	Description
Therapist	Person who provides therapy services to clients.
Client	Person who seeks therapy services

Email(Postmark)	Service which sends emails
SMS(MSG91)	Service which sends SMS, especially login OTP
Supabase	Service that hosts Postgres database and provides real-time services, which are used for chat and implementing outbox pattern.
Jitsi	Self-hosted video conference service

3.2. Technical Context

The following shows how Emotist is deployed and it's important components.



Name	Description
------	-------------

Therapist Web App	ReactJS web application used by therapists.
Client Mobile App	React Native mobile application used by clients.
AWS NLB	AWS managed Network Load Balancer(NLB), which receives the traffic from public internet and sends it to the EKS cluster.
Jisti	AWS EC2 machine that runs the open-source Jisti in Docker container
External Gateway	Envoy gateway(Gateway API), which is responsible for routing requests to the appropriate services
Therapist HTTP Route	HTTP route configuration, which routes the traffic to the Therapist service
API HTTP Route	HTTP route configuration, which routes the traffic to the API service
Client HTTP Route	HTTP route configuration, which routes the traffic to the Client service
Therapist	Service which hosts therapist web application.
API	Service which hosts backend api application.
Client	Service which hosts client web application.
Redis Cluster	Redis cluster running in sentinel topology, used by the BullMQ(messaging queue)
Supabase	Database and real-time service provider
Postmark	Emailing service provider
MSG91	SMS service provider

4. Solution Strategy

This section outlines the key strategic decisions that influence the architecture of the **Emotist** backend API, covering aspects of structure, technology, integration, and quality goals.

4.1. Domain-Driven Design (DDD)

The system is organised using **Domain-Driven Design**, with clearly defined **bounded contexts** such as:

Bounded Context	Description
Therapist	Managing therapist onboarding, profile, and client relationships
Client	Handling client onboarding, booking/managing appointments, and personal data
Appointment	Responsible for scheduling appointments, checking availability of time slots of a service provider.

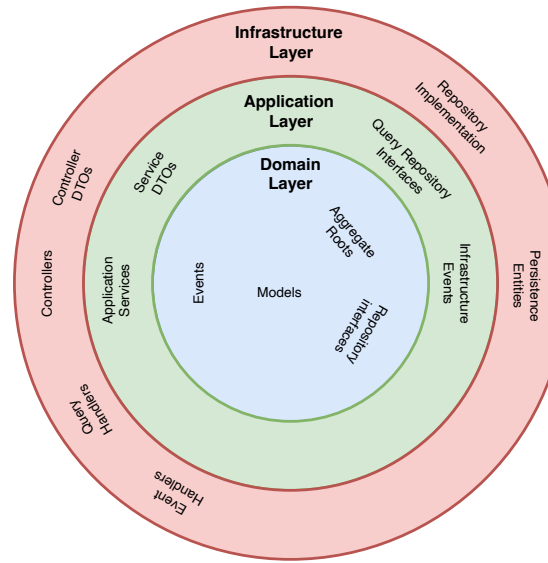
Each context encapsulates its own domain logic, aggregates, and persistence models, promoting modularity and team autonomy.

4.2. Clean Architecture

The codebase follows **Clean Architecture** principles to enforce separation of concerns between:

- **Domain layer** – pure business logic and aggregates
- **Application layer** – use cases and coordination logic
- **Infrastructure layer** – frameworks, database access, and external APIs

This allows testability, maintainability, and independent evolution of the system's layers.



4.3. Integration and Communication

- Internal communication between bounded contexts is **event-driven**, ensuring loose coupling and eventual consistency.
- External integrations (e.g., payment gateway, email, SMS) are encapsulated behind infrastructure interfaces to reduce vendor lock-in.
- Queries that span multiple domains are handled via **query repositories** and **application-level composition**.

4.4. Security and Privacy

- All communication is encrypted via HTTPS and protected using tokens (JWT)
- Access control and authorisation checks are enforced on the infrastructure layer.
- Row Level Security(RLS) for protecting data access.

4.5. Deployment Strategy

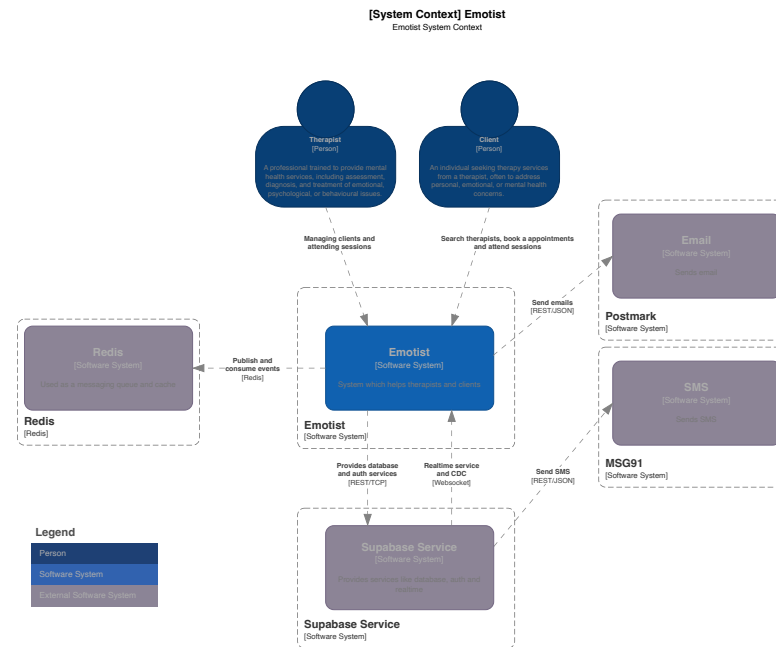
The system is containerised using Docker and deployed on EKS, enabling:

- Horizontal scaling of services
- Environment-specific configurations
- Rolling updates with zero downtime

5. Building Block View

5.1. Context Diagram

The context diagram provides a high-level overview of the **Emotist** system and its interactions with external actors and systems, defining the system boundaries and clarifying responsibilities and communication paths.



5.1.1. Emotist System Role

Emotist is the core software system that connects **therapists** and **clients**, enabling mental health service delivery through features like appointment management, client onboarding, real-time communication, etc...

5.1.2. Primary Users (Actors)

- **Therapist:** A professional who uses Emotist to manage client relationships, attend therapy sessions, and track progress.
- **Client:** An individual seeking therapy services. They use Emotist to search for therapists, book sessions, and participate in online consultations.

5.1.3. External Systems and Interfaces

External System	Purpose
Redis	Used as a messaging queue and cache , enabling event-driven communication within and across bounded contexts.
Supabase Service	Provides authentication, database, and real-time sync services via REST, WebSockets, and TCP.
Postmark (Email)	Sends transactional emails such as invitations, reminders, and notifications.
MSG91 (SMS)	Sends transactional SMS messages like OTPs and session alerts.

5.1.4. Interaction Overview

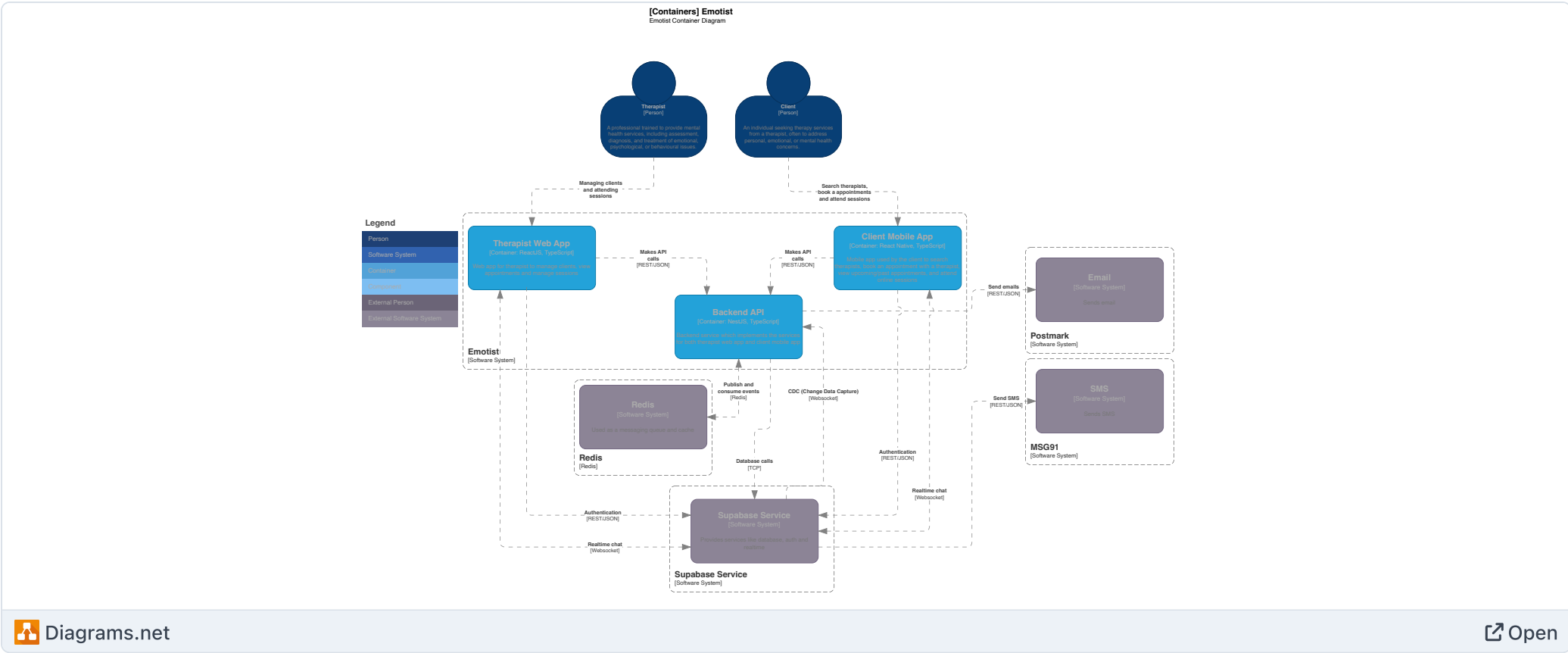
- Therapists manage clients and sessions via Emotist’s backend API.
- Clients interact with Emotist to discover therapists, book appointments, and join sessions.
- Emotist interacts with **Redis** to **publish/consume domain events**.
- Emotist uses **Supabase** for **authentication, data persistence, and realtime communication**.
- Email and SMS are sent via **Postmark** and **MSG91**, respectively, using REST/JSON interfaces.

5.1.5. Communication Protocols

- REST and WebSocket APIs for frontend and mobile app integration
- Redis pub/sub for internal message/event handling
- REST-based calls for sending notifications (email/SMS)
- WebSocket for real-time updates to clients and therapist (via Supabase)

5.2. Container Diagram

The container diagram shows the main building blocks of the **Emotist** system and how responsibilities are distributed across different containers. It illustrates how the system fulfils functional requirements and interacts with external systems and users.



5.2.1. Containers in Emotist

Container	Technology	Responsibility
Therapist Web App	ReactJS, TypeScript	Web interface for therapists to manage clients, appointments, and sessions.
Client Mobile App	React Native, TypeScript	Mobile app for clients to find therapists, book sessions, and attend appointments.
Backend API	NestJS, TypeScript	Central service that processes business logic, handles HTTP requests, and coordinates communication with external services and data stores.

Redis	Redis	Used for message queues and caching; supports event-driven architecture (via BullMQ).
Supabase Service	Supabase	Provides authentication, database storage, and real-time features like chat and CDC via WebSocket.
Postmark (Email)	REST/JSON	Sends transactional and notification emails to clients and therapists.
MSG91 (SMS)	REST/JSON	Sends SMS for OTPs, alerts, and reminders.

5.2.2. User Interaction Flows

- **Therapist** uses the **Web App** to manage their schedule, view session notes, and interact with clients. The web app communicates with the **Backend API** via REST.
- **Client** uses the **Mobile App** to search therapists, view upcoming appointments, and attend therapy sessions. The mobile app also communicates with the **Backend API** over REST.

5.2.3. Internal Communication

- **Backend API** interacts with:
 - **Redis** to **publish and consume events** for cross-bounded context communication.
 - **Supabase** for **authentication, database operations**, and **WebSocket-based real-time features** like chat and data sync.
 - **Postmark** and **MSG91** via REST to send email and SMS notifications respectively.

5.2.4. Integration Protocols

Communication	Description
REST/JSON	Used between frontend apps and backend, as well as for external notifications (email/SMS).
WebSocket	Used for real-time updates (e.g., chat, CDC) via Supabase.
Redis Events	Backend modules communicate asynchronously via publish-subscribe event model using Redis.

TCP	Backend API accesses the database layer in Supabase using traditional database connections.
-----	---

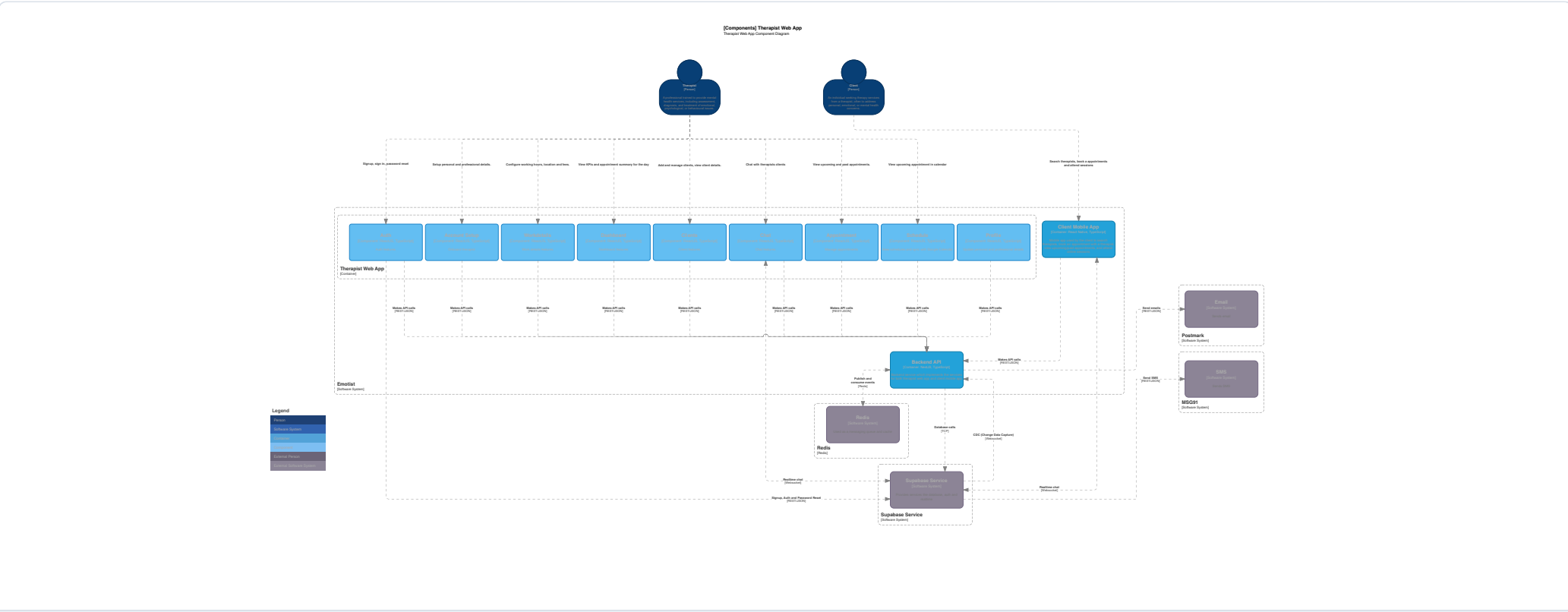
5.2.5. Key Architectural Considerations

- Backend is responsible for enforcing business rules and orchestrating workflows across bounded contexts.
- Real-time communication is abstracted through Supabase, which simplifies implementation of features like presence, chat, and instant updates.
- The use of Redis and event publishing allows the system to scale and decouple processing between bounded contexts.

5.3. Component Diagram

5.3.1. Therapist Web App

The **Therapist Web App** is a React-based frontend container designed specifically for therapists to manage their profile, clients, and therapy sessions. It is divided into modular components that each address a specific feature of the platform, promoting separation of concerns and reusability.



Key Components

Component	Purpose
Auth	Handles signup, login, and password reset via Supabase Auth integration.
Account Setup	Guides therapists through personal and professional onboarding.
Workdetails	Enables configuration of working hours, fees, and availability.
Dashboard	Provides a summary view of sessions, upcoming tasks, and calendar info.
Clients	Manages client list, client profiles, and interaction history.
Chat	Facilitates secure, real-time messaging between therapists and clients.
Appointment	Enables session management including booking, rescheduling, and cancelling.
Schedule	Visualises therapist availability and integrates with calendar views.
Profile	Allows editing of personal and professional therapist information.

Communication

- All components communicate with the **Backend API** via **REST/JSON** calls.
- Auth operations rely on **Supabase** (via REST).
- Real-time features like **chat** use **WebSocket** via Supabase Realtime.

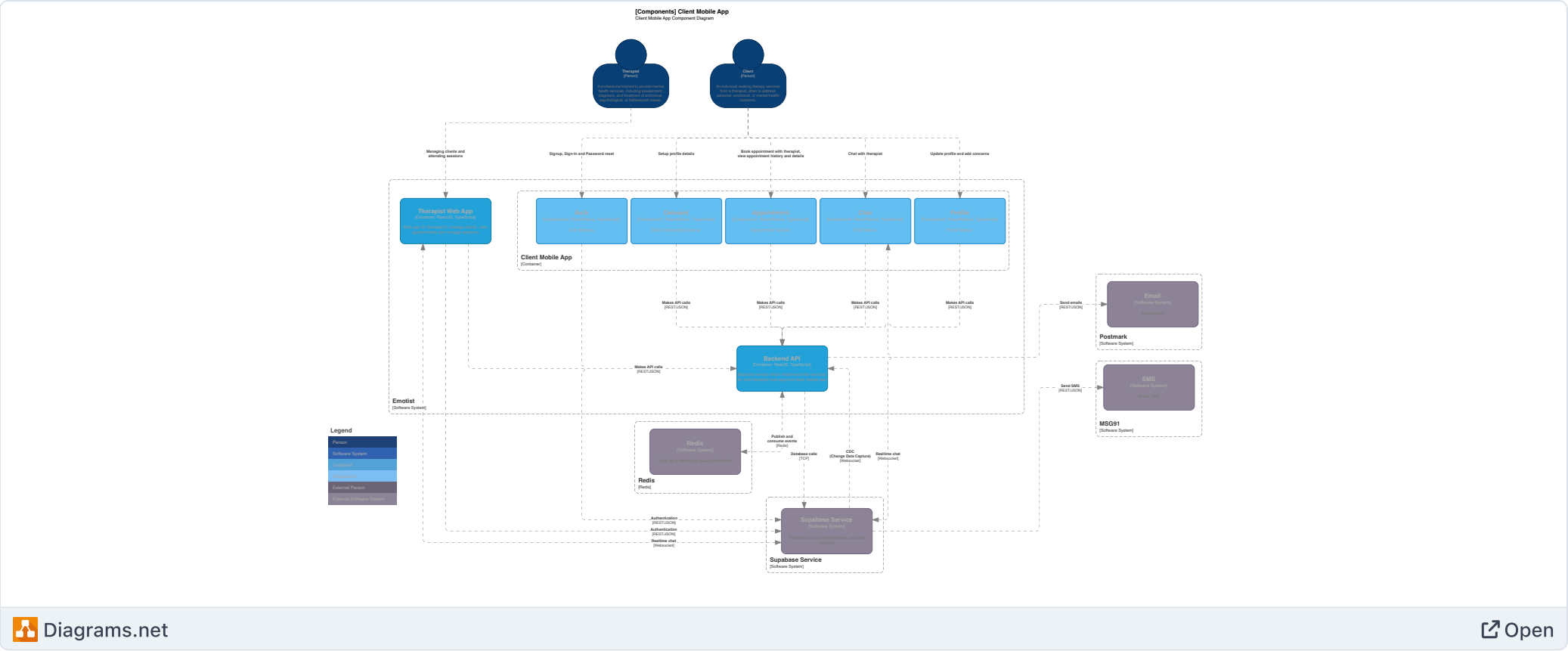
Design Considerations

- Each component is built in **ReactJS + TypeScript** and follows a clear domain boundary aligned with backend bounded contexts.
- Modular component design allows independent development, testing, and reuse across therapist workflows.
- Emphasis is placed on UX to streamline therapist interactions and minimize friction in day-to-day tasks.

5.3.2. Client Mobile App

The **Client Mobile App** is a React Native–based container designed for clients seeking mental health support. It allows clients to search for therapists, book and attend appointments, communicate with therapists, and manage their profile.

The app is organised into modular components, each built using **React Native + TypeScript**, enabling a consistent codebase and clear separation of features.



Key Components

Component	Purpose
Auth	Manages signup, login, and password reset using Supabase Auth.
Onboard	Guides clients through the onboarding process, including profile setup.
Appointment	Enables clients to book, view, and manage appointments with therapists.
Chat	Provides real-time secure messaging between client and therapist.
Profile	Allows clients to update personal details and specify therapy-related concerns.

Communication

Key Components (Bounded Contexts)

Component	Purpose
Therapist Bounded Context	Manages therapist-specific functionality like onboarding, profile, availability, and client relationships.
Client Bounded Context	Handles client profile, onboarding status, and concerns.
Appointment Bounded Context	Manages appointment booking, rescheduling, cancellation, and therapist availability.
Infrastructure	Provides technical concerns like persistence, messaging, real-time sync, and external integrations.

Communication

- The **Therapist Web App** and **Client Mobile App** communicate with the backend using **REST/JSON**.
- Internally, the bounded contexts query each other through defined interfaces (query services).
- Events are published and consumed asynchronously using **Redis Pub/Sub**, enabling decoupled interactions between bounded contexts.
- **Postmark** and **MSG91** are used by infrastructure components to send email and SMS notifications respectively.
- **Supabase** provides the database layer and authentication services, accessed via REST and TCP (for queries).

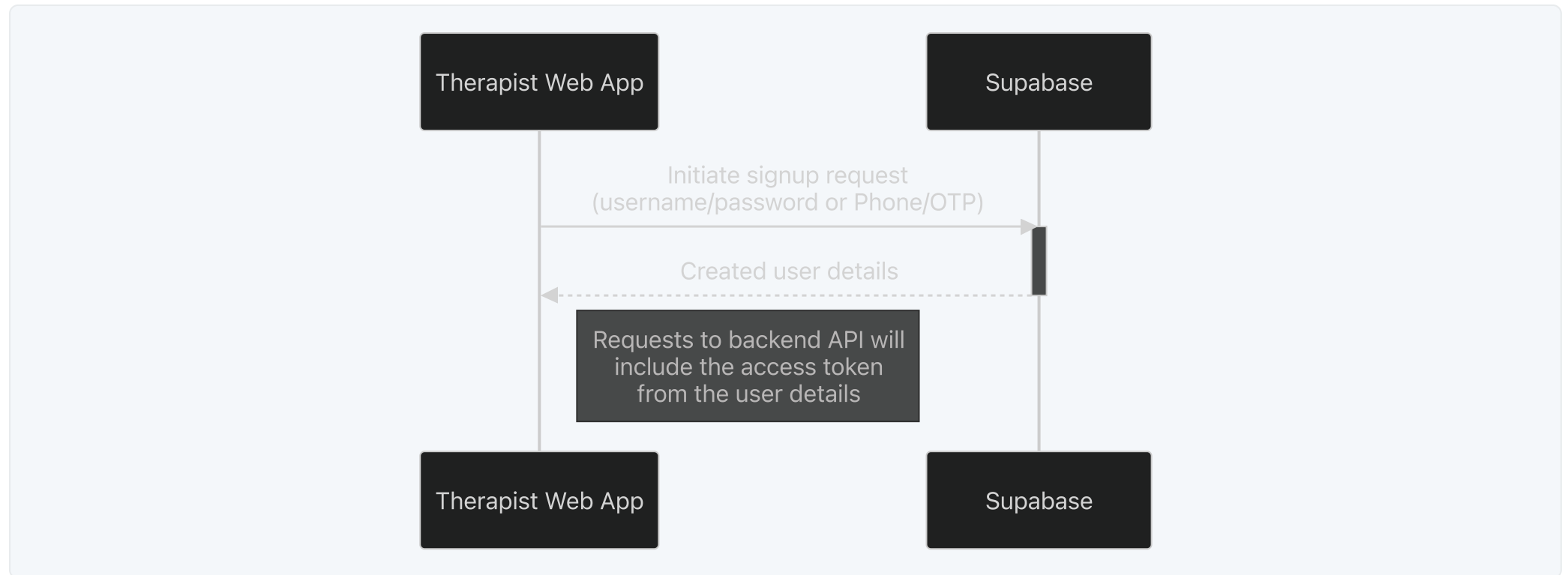
Design Considerations

- The architecture ensures **strong modularity**, with each bounded context owning its own domain logic and persistence concerns.
- The **Infrastructure component** encapsulates third-party integrations and shared cross-cutting concerns (e.g., email, SMS, real-time updates).
- The use of **asynchronous events via Redis** promotes scalability and cross-context decoupling.
- Backend services are designed to be **stateless**, with authentication and real-time presence handled by **Supabase**.

6. Runtime View

The runtime view describes how the Emotist system's components collaborate during key use cases. Typical runtime scenarios involve the **client or therapist interacting with the frontend**, which communicates with the **Backend API** over REST. The backend delegates to relevant **bounded contexts** (e.g., Therapist, Appointment) and may publish **events via Redis** for asynchronous processing.

6.1. Therapist Signup

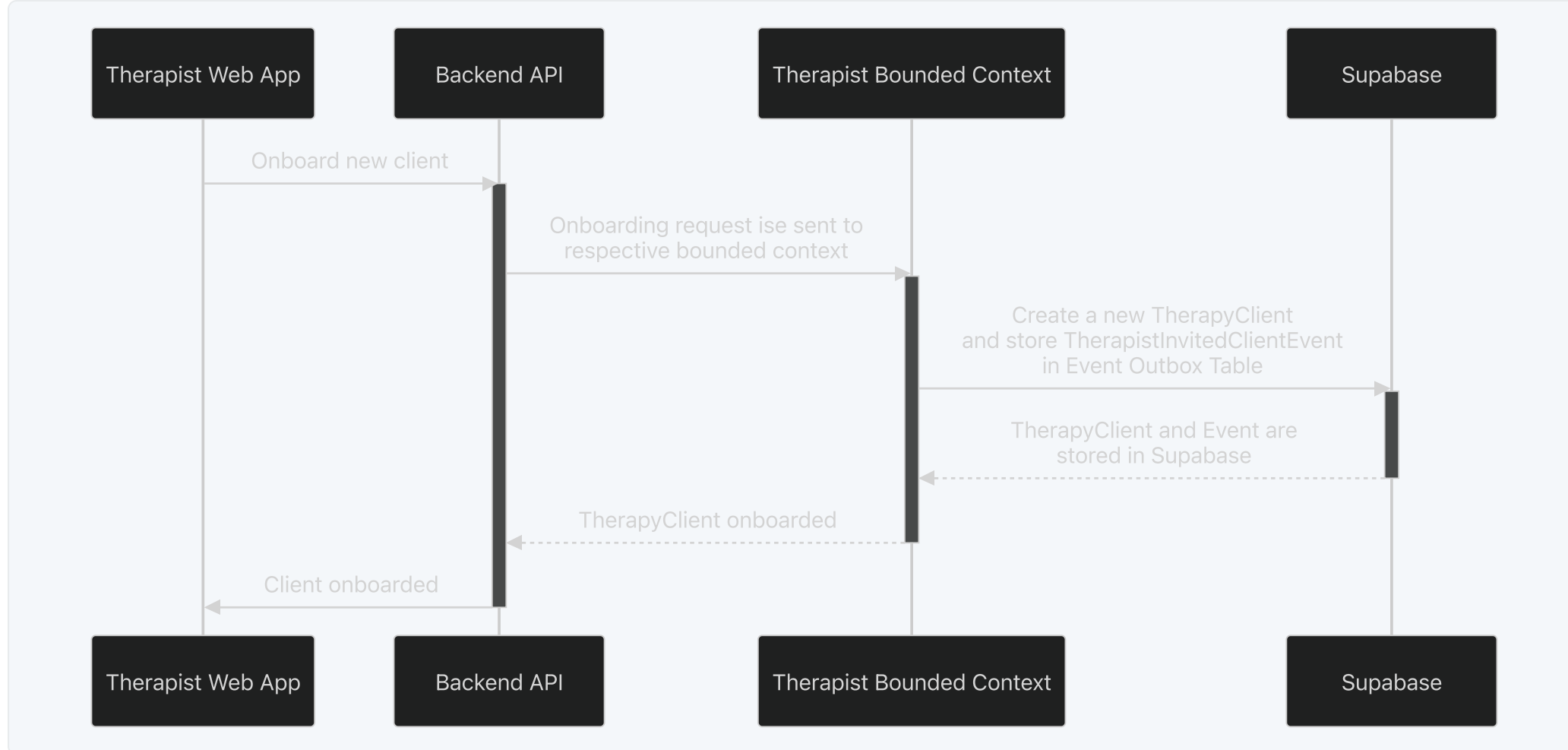


In the therapist signup flow, the **Therapist Web App** initiates the registration by sending a signup request directly to **Supabase**, which handles authentication (via username/password or phone/OTP). Supabase creates the user account and returns the user details along with an **access token**. Subsequent requests from the frontend to the **Backend API** include this token for authorisation. No backend service is directly involved in the initial signup; it relies entirely on Supabase for identity provisioning.

6.2. Therapist Client Invitations

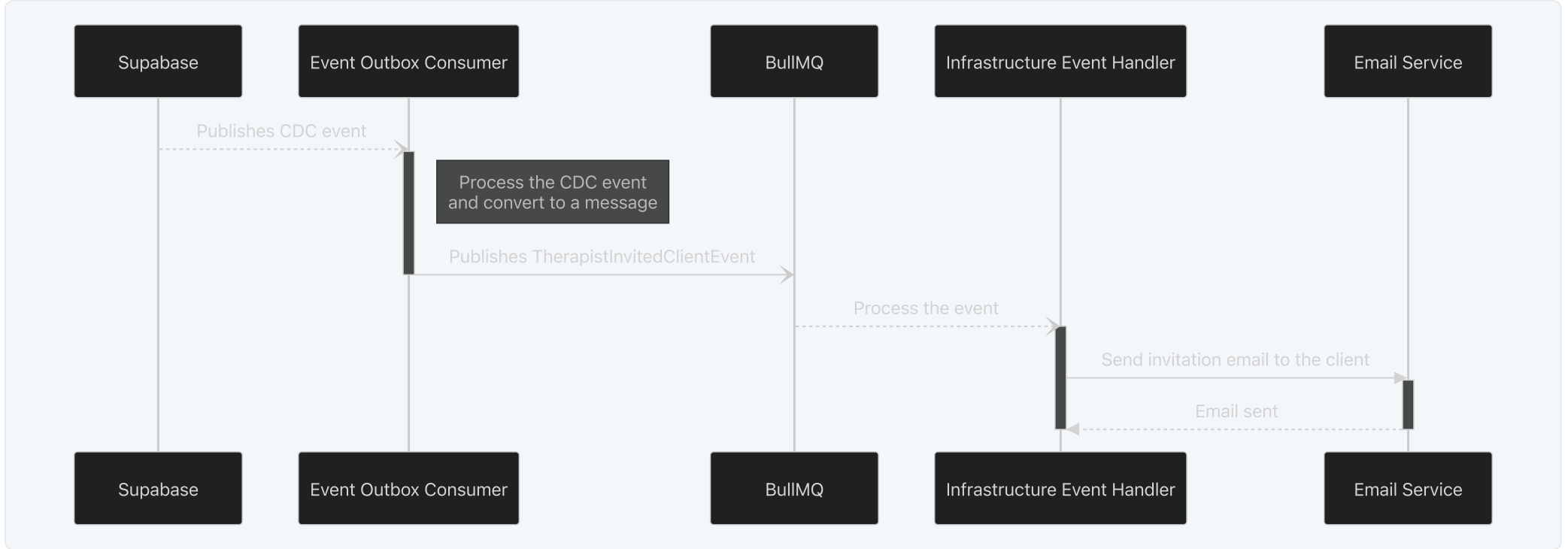
Use case describing how a therapist invites clients.

6.2.1. Therapist Inviting Client



When a therapist initiates the onboarding of a new client via the **Therapist Web App**, the request is sent to the **Backend API**, which delegates it to the **Therapist Bounded Context**. The bounded context handles the domain logic by creating a new `TherapyClient` entity and recording a `TherapistInvitedClientEvent` in the **event outbox table**. Both the entity and the event are persisted via **Supabase**. Once stored, control flows back through the API to the frontend, confirming that the client has been successfully onboarded. This event can then be published asynchronously for cross-context actions.

6.2.2. Processing TherapistInvitedClientEvent

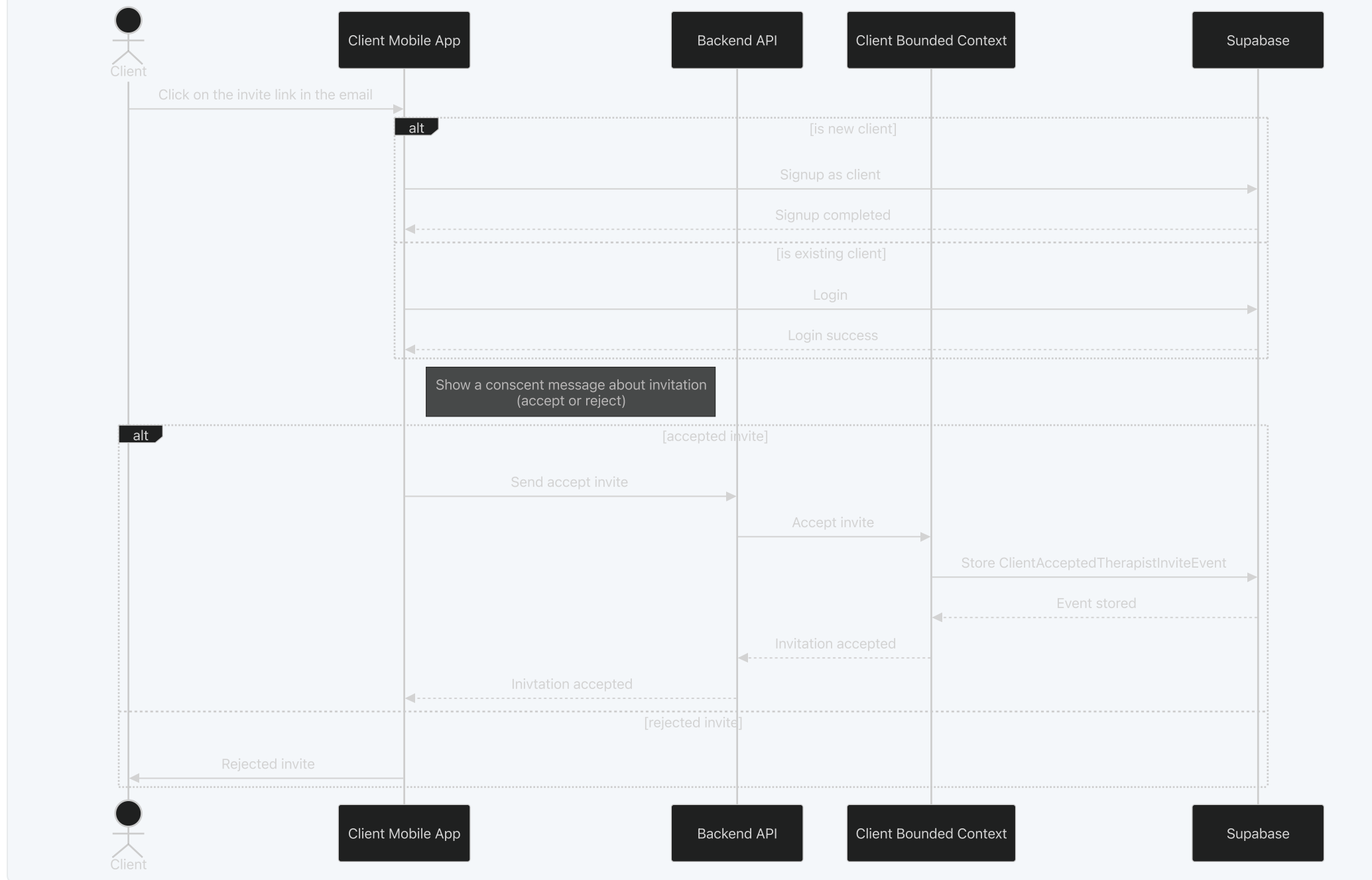


Once the `TherapistInvitedClientEvent` is recorded in the **Event Outbox Table**, Supabase emits a **Change Data Capture (CDC)** notification. This triggers the **Event Outbox Consumer (EOC)**, which listens to CDC events, transforms the row-level change into a domain event, and publishes it to **BullMQ**.

The event is picked up by the **Infrastructure Event Handler**, which reacts to the `TherapistInvitedClientEvent` by invoking the **Email Service** to send an invitation email to the client. Once the email is sent, the event is considered fully processed.

This flow ensures **eventual consistency** between domain actions and external side effects (emailing), leveraging **CDC + outbox pattern** for reliable event publishing.

6.2.3. Client Accepting Invitation



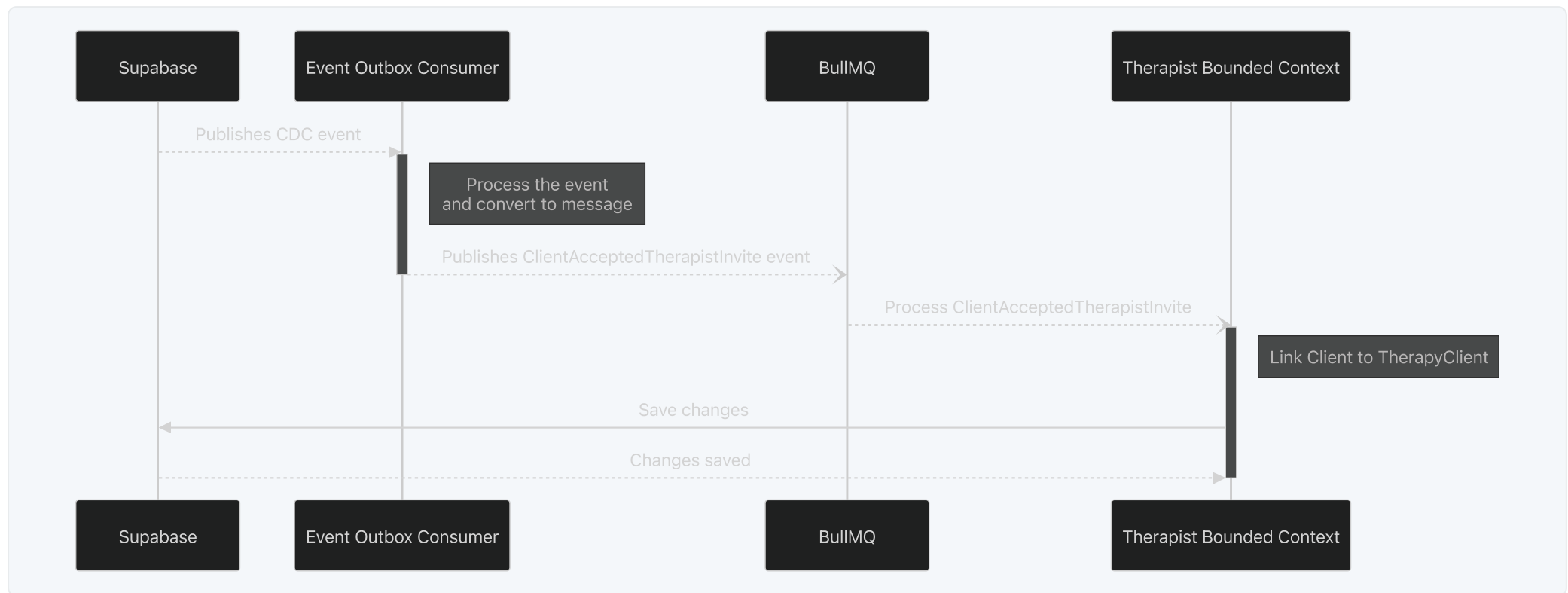
When a client clicks the invitation link from their email, they are redirected to the **Client Mobile App**. Depending on their status, they either **sign up** as a new user or **log in** if they already have an account—both actions are handled via **Supabase Auth**.

After authentication, the app presents a **consent screen** allowing the client to either **accept** or **reject the therapist's invitation**.

- If the client **accepts**, the app sends a request to the **Backend API**, which forwards it to the **Client Bounded Context**. The domain logic records a `ClientAcceptedTherapistInviteEvent`, which is stored via **Supabase** (event outbox pattern).
- If the client **rejects**, no backend action is taken, and the user is informed.

This flow ensures that therapist-client associations are formed explicitly with the client's consent and are tracked via domain events for downstream processing.

6.2.4. Linking Client With TherapyClient



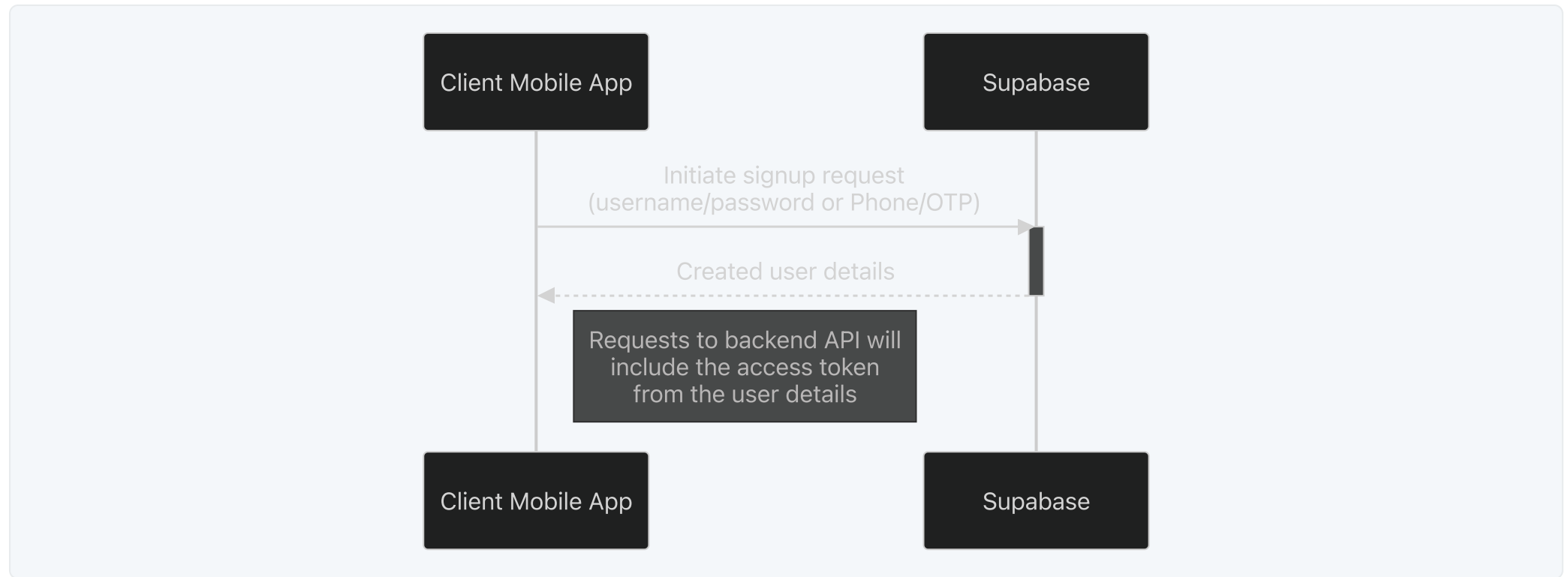
When a client accepts an invitation, a `ClientAcceptedTherapistInviteEvent` is written to the **event outbox** in the **Client Bounded Context** and persisted in **Supabase**. This triggers a **Change Data Capture (CDC)** event, which is consumed by the **Event Outbox Consumer (EOC)**.

The EOC processes the CDC event, transforms it into a domain event, and publishes it to **BullMQ**. The event is then consumed by the **Therapist Bounded Context**, which executes the domain logic to **link the client (from Client BC) with the corresponding TherapyClient entity** (in Therapist BC).

This is persisted back into **Supabase**, completing the association between the two representations of the client across bounded contexts.

This pattern supports **event-driven consistency** and keeps both domains synchronised while maintaining clear separation of concerns.

6.3. Client Signup

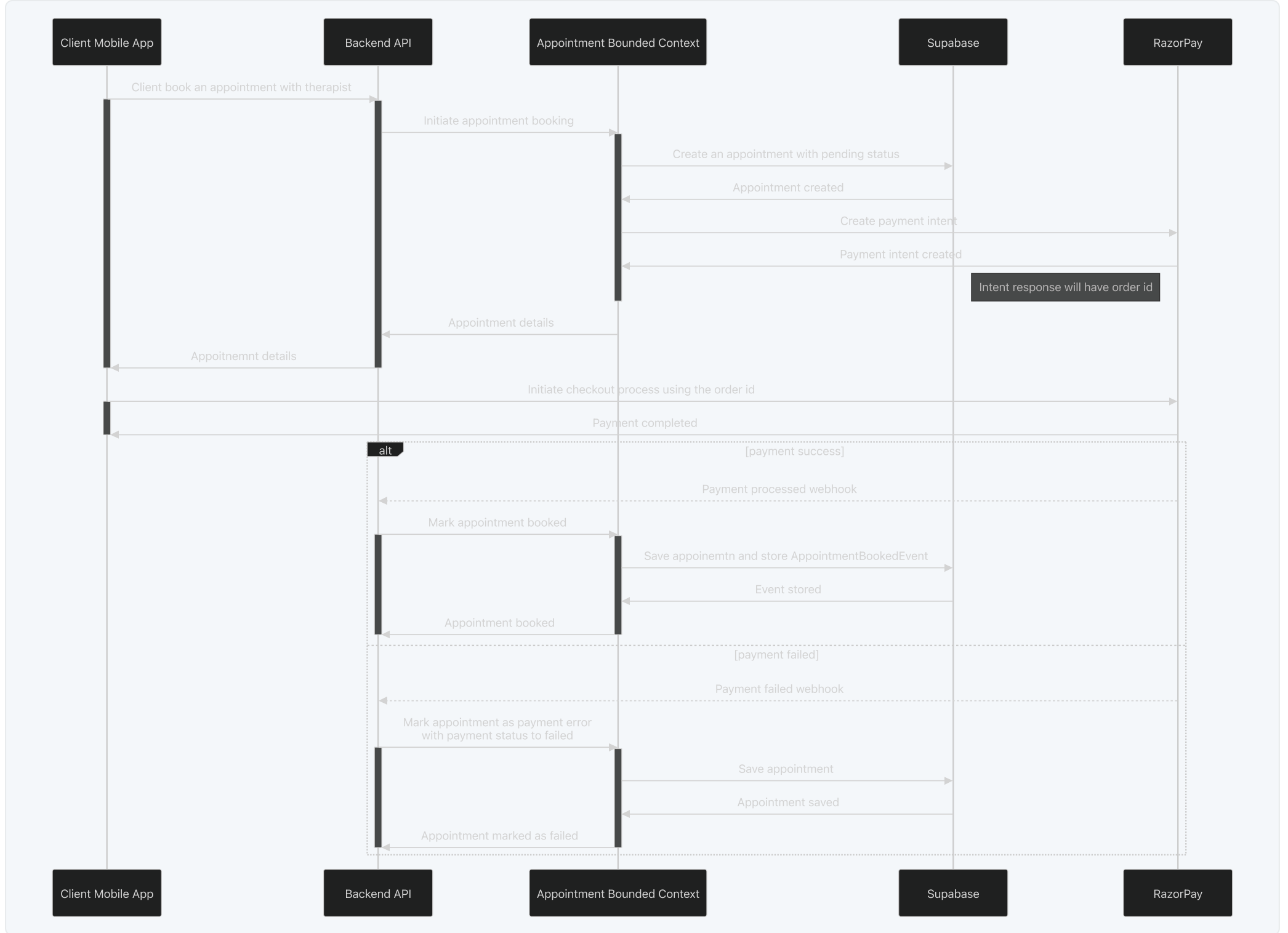


In the client signup flow, the **Client Mobile App** directly interacts with **Supabase** to register a new user using either email/password or phone/OTP authentication. Supabase handles the identity creation and returns the authenticated user details along with an **access token**. The mobile app stores this token and includes it in subsequent requests to the **Backend API** for authorised access to client-specific features. The backend is not involved in the signup process itself but relies on the token for identity verification.

6.4. Client Booking an Appointment

Use case describing how a client books an appointment with a therapist.

6.4.1. Client Book an Appointment from Mobile App



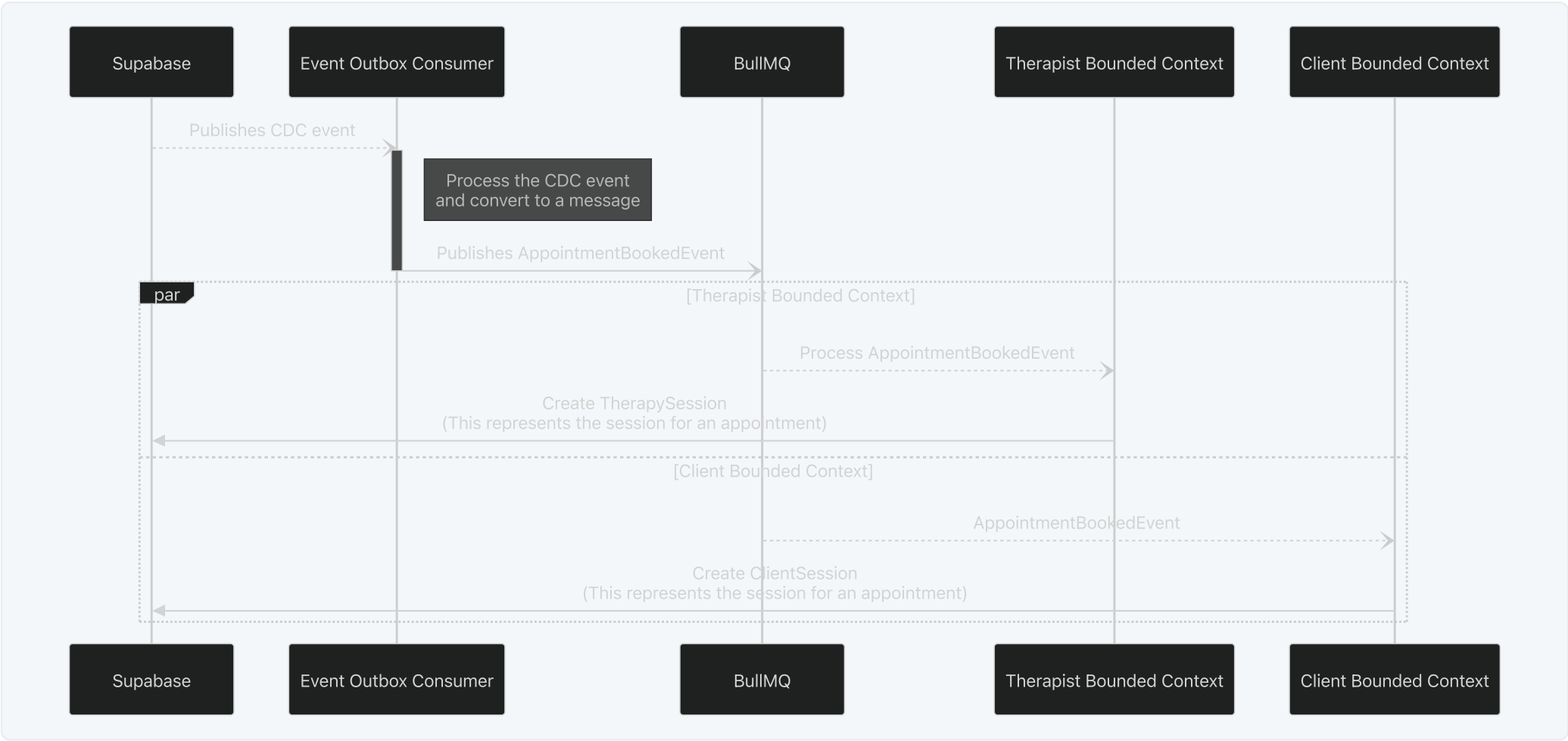
When a client initiates an appointment booking via the **Client Mobile App**, the request is sent to the **Backend API**, which delegates the action to the **Appointment Bounded Context**. The context creates an appointment in **Supabase** with a **pending** status, and then contacts **Razorpay** to generate a **payment intent** (with an order ID). These appointment and payment details are returned to the mobile app.

The client proceeds to complete the payment through Razorpay using the order ID. Razorpay sends a **webhook callback** to the **Backend API** with the payment result:

- On **payment success**, the appointment is marked as **booked** , and an **AppointmentBookedEvent** is recorded in Supabase.
- On **payment failure**, the appointment is updated with a failed payment status.

This flow ensures that **appointments are confirmed only after successful payment**, maintaining transactional integrity between booking and billing.

6.4.2. Creating ClientSession and TherapySession on Appointment Booking



After a client successfully books an appointment, an `AppointmentBookedEvent` is stored in the **Supabase** event outbox. This triggers a **Change Data Capture (CDC)** event that is consumed by the **Event Outbox Consumer (EOC)**, which converts it into a message and publishes it to **BullMQ**.

The event is then processed in **parallel** by:

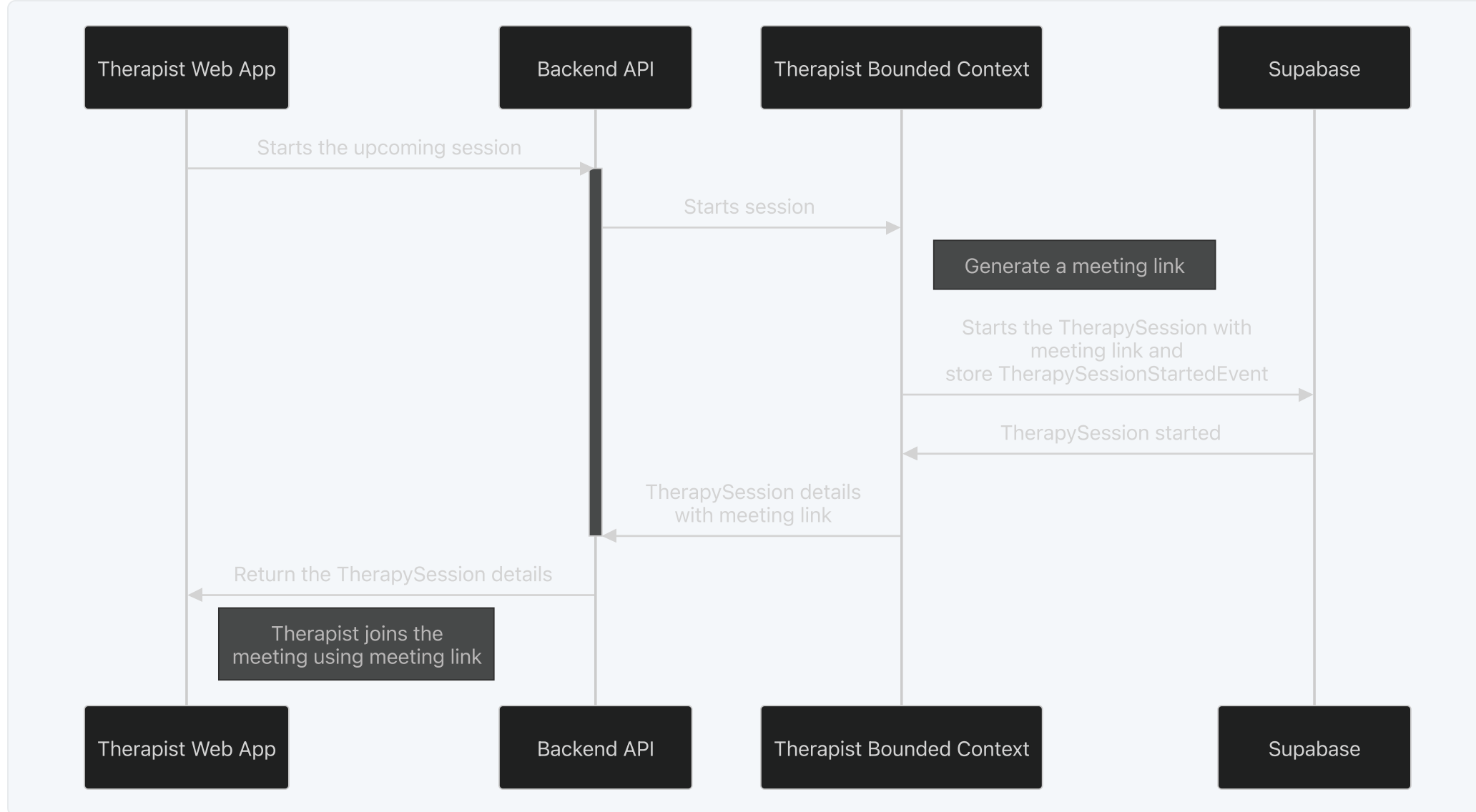
- The **Therapist Bounded Context**, which creates a corresponding `TherapySession` entity to represent the session from the therapist's perspective.
- The **Client Bounded Context**, which creates a `ClientSession` entity to represent the same appointment from the client's point of view.

Both records are persisted in **Supabase**, maintaining bounded context separation while ensuring a consistent view of the appointment. The `TherapySession` exists only within the **therapist context**, and the `ClientSession` only within the **client context**—supporting autonomy, encapsulation, and asynchronous consistency through event-driven design.

6.5. Therapist Starting a Session

Use cases describing how a therapist starts an online session and how clients get the meeting invite link.

6.5.1. Therapist Starts a Session



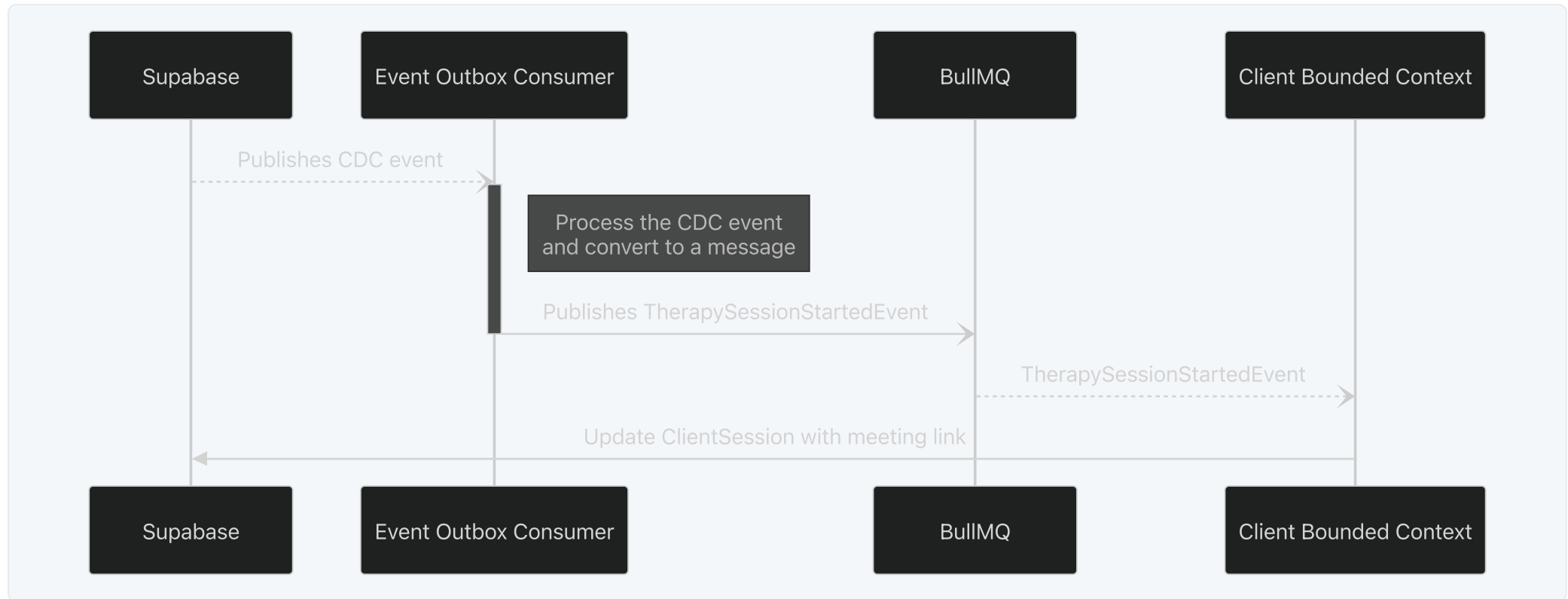
When a therapist chooses to start an upcoming session from the **Therapist Web App**, a request is sent to the **Backend API**, which delegates the operation to the **Therapist Bounded Context**.

The bounded context performs domain logic to **generate a meeting link** (e.g., for Jitsi) and **starts the `TherapySession`**, updating its status and storing a `TherapySessionStartedEvent` in **Supabase**.

Once the session is started and persisted, the API returns the updated `TherapySession` details—including the meeting link—back to the frontend. The therapist then uses this link to join the live session.

This process ensures that session activation and meeting setup are **controlled by domain rules** and tracked through events for consistency and traceability.

6.5.2. Update ClientSession with meeting link



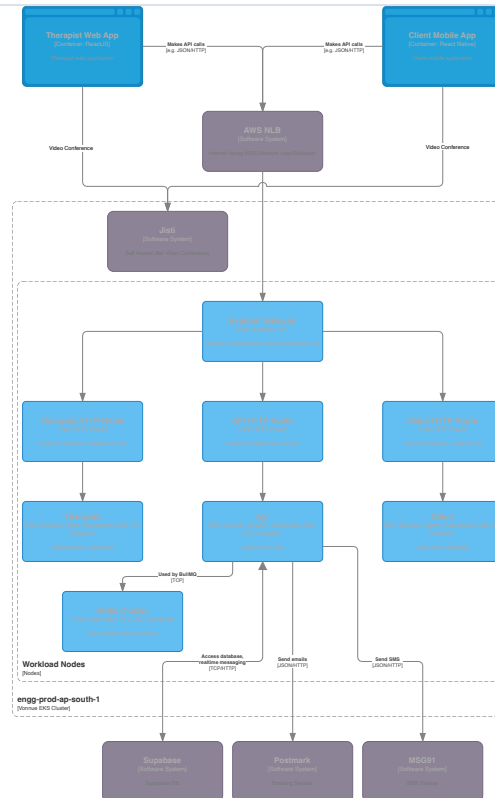
When a therapist starts a session, a `TherapySessionStartedEvent` is recorded in the **Therapist Bounded Context** and stored in **Supabase**. This triggers a **Change Data Capture (CDC)** event, which is picked up by the **Event Outbox Consumer (EOC)**.

The EOC processes the CDC event, transforms it into a domain event, and publishes the `TherapySessionStartedEvent` to **BullMQ**. The event is then consumed by the **Client Bounded Context**, which reacts by updating the corresponding `ClientSession` in **Supabase** with the generated **meeting link**.

This event-driven update ensures the client has access to the correct meeting link without creating a tight dependency between bounded contexts, maintaining **loose coupling and eventual consistency**.

7. Deployment View

The Emotist system is deployed on a **Kubernetes cluster (EKS)** in the AWS Mumbai region ([ap-south-1](#)). External traffic is routed through an **AWS Network Load Balancer (NLB)**, which forwards requests to an **External Gateway (Envoy Gateway)** responsible for dispatching API calls to appropriate internal services.



Core Application Pods

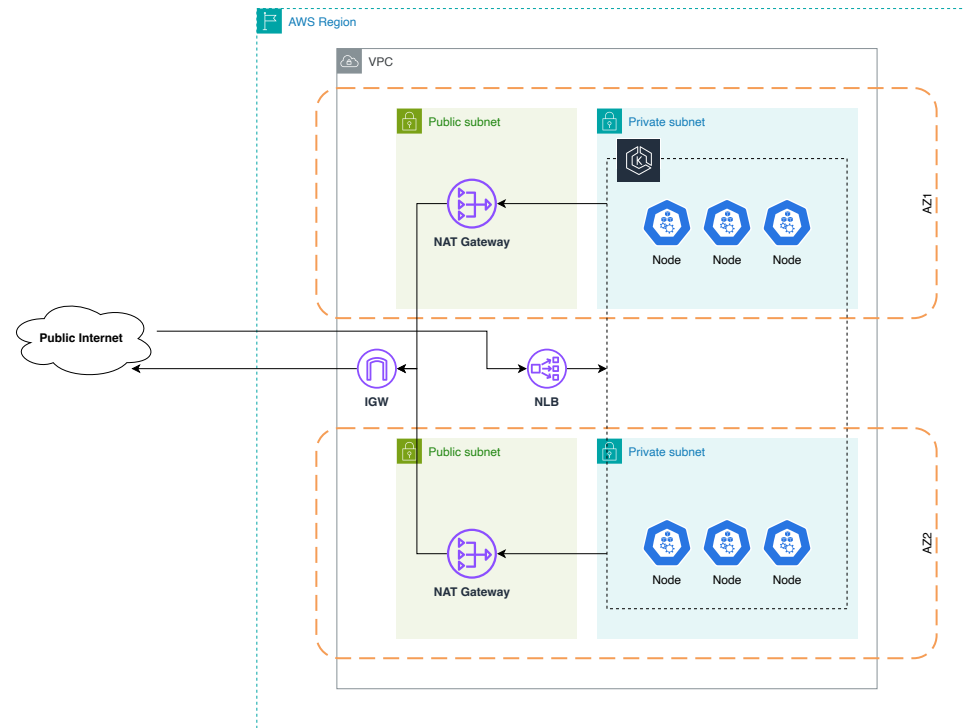
- **Therapist**, **Client**, and **API** pods handle web frontend, mobile API, and backend logic respectively.
- All pods are containerised and deployed using **OCI-compliant containers**.
- A **Redis Cluster** is deployed as a Kubernetes pod to support message queues for **BullMQ**.

External Dependencies

- **Supabase** provides authentication, database, and real-time messaging over TCP and WebSocket.
- **Postmark** and **MSG91** are used for sending emails and SMS via JSON/HTTP.
- A self-hosted **Jitsi** server is used for video conferencing between therapists and clients.

This deployment architecture ensures scalability, separation of concerns, and secure communication across services.

7.1. Infrastructure



The Emotist platform is hosted on an **Amazon EKS cluster** deployed across two **Availability Zones (AZs)** for high availability and fault tolerance. The cluster resides within a **VPC** that is segmented into **public and private subnets**:

- **Private Subnets** contain Kubernetes **worker nodes**, which run application workloads (e.g., API, frontend, Redis).
- **Public Subnets** host the **NAT Gateways** (for outbound internet access from private nodes) and the **AWS Network Load Balancer (NLB)**, which routes external traffic into the cluster.
- An **Internet Gateway (IGW)** enables ingress/egress from the public internet.

The architecture ensures that all core application services run in isolated **private subnets**, while public endpoints are securely managed through the **NLB and gateway routing**. This design promotes both **network-level isolation** and **scalability** for cloud-native workloads.

8. Cross-Cutting Concepts

The Emotist system applies several cross-cutting concepts that influence multiple parts of the architecture:

- **Authentication & Authorisation:** Handled via **Supabase Auth**, using JWT-based tokens passed from the frontend to the backend. Role-based access control is enforced on every request.
- **Event-Driven Architecture:** Bounded contexts communicate through asynchronous **domain events** published to **BullMQ** via an **event outbox + CDC pattern**, ensuring loose coupling and eventual consistency.
- **Observability:** The system uses **OpenTelemetry** for distributed tracing, request logging, and performance monitoring across backend services.
- **Error Handling:** Consistent error handling is implemented at the domain, application service and controller layers, with proper error mapping on each layer.
- **Localisation:** The frontend supports multiple languages via **i18next**, with localised values served based on the request context.

These concepts are implemented consistently across the system to promote maintainability, scalability, and a cohesive developer experience.

9. Architecture Decisions

9.1. Use Domain-Driven Design (DDD)

Date	Oct 7, 2024
Created By	@ Rajeesh CV
Status	Accepted
Context	The platform involves distinct areas like therapist onboarding, client engagement, and appointment scheduling, each with its own domain logic and lifecycle.
Decision	Structure the backend using DDD with multiple bounded contexts (e.g., Therapist, Client, Appointment).
Consequences	• Enables clear separation of concerns and modular development. • Facilitates team ownership of features. • Introduces complexity in cross-context communication (handled via events).

9.2. Adopt Clean Architecture with Layered Structure

Date	Oct 7, 2024
Created By	@ Rajeesh CV
Status	Accepted
Context	Business rules must remain independent of frameworks and infrastructure.
Decision	Implement each bounded context using Clean Architecture , separating domain, application, and infrastructure layers.
Consequences	- Improves testability and long-term maintainability. - Slower initial development due to layered abstraction. - Requires discipline to avoid shortcutting layers.

9.3. Implement Event-Driven Communication via Redis and BullMQ

Date	Oct 7, 2024
Created By	@ Rajeesh CV
Status	Accepted
Context	Bounded contexts must react to changes in other parts of the system without tight coupling.
Decision	Use Redis (BullMQ) for asynchronous event publishing and consumption, combined with event outbox + CDC pattern .
Consequences	- Promotes loose coupling and eventual consistency. - Using BullMQ reduces the cost of having a separate messaging queue. - Adds infrastructure complexity (Redis, CDC, outbox management). - Introduces debugging and traceability challenges across services.

9.4. Use OpenTelemetry for Observability

Date	Oct 7, 2024
Created By	@Rajeesh CV
Status	Accepted
Context	Distributed components and asynchronous events require traceability and performance monitoring.
Decision	Integrate OpenTelemetry for distributed tracing, metrics, and structured logging.
Consequences	- Provides deep insight into system performance and event flows. - Requires instrumentation and consistent span propagation. - Adds some overhead to request processing.

9.5. Integrate Razorpay for Payment Flow

Date	Oct 7, 2024
Created By	@Rajeesh CV
Status	Accepted
Context	Platform requires secure and localised payment handling for session bookings.
Decision	Use Razorpay for payment intent generation, checkout flow, and webhook-based confirmation.
Consequences	- Reduces time to implement and secure payment processing. - Relies on external webhook reliability. - Requires mapping payment lifecycle to domain model carefully.

9.6. Use a Design System and Build a Shared Component Library for Web Frontend

Date	Oct 7, 2024
------	-------------

Created By	@ Rajeesh CV
Status	Accepted
Context	All the web App needs a consistent, accessible, and maintainable UI across multiple features and screens. Designers and developers need a shared visual language to avoid duplication and improve UX consistency.
Decision	Adopt a design system and develop a shared React component library that encapsulates design tokens, reusable UI components, and layout primitives. The library is versioned and maintained as a separate package in the monorepo.
Consequences	• Ensures visual consistency and design scalability across all frontend apps. • Reduces rework and accelerates feature development through reusable UI building blocks. • Introduces initial setup and ongoing maintenance overhead for the design system and library. • Requires close collaboration between design and frontend engineering teams to evolve the system responsibly.

10. Quality Requirements

This section outlines the key quality attributes that the Emotist system must fulfil to meet stakeholder expectations and ensure long-term success. These qualities guide architectural decisions and trade-offs throughout the system's lifecycle.

10.1. Quality Tree

The quality tree classifies and prioritises quality attributes such as **maintainability**, **scalability**, **security**, **usability**, and **observability**. It helps visualise which attributes are most critical to the success of the platform and identifies areas that require focused attention during design and implementation.

10.2. Quality Scenarios

Quality scenarios define concrete, measurable situations that illustrate how the system should behave under specific conditions. These scenarios help validate whether the architecture supports the intended quality goals, such as:

- Handling increasing therapist onboarding volume (**scalability**)
- Updating session logic without breaking client features (**modifiability**)
- Tracking session failures across services (**observability**)
- Ensuring secure access to sensitive data (**security**)

11. Risks and Technical Debts

This section identifies known **technical risks** and **areas of technical debt** within the Emotist platform that could affect future maintainability, scalability, or reliability. Proactively recognizing these challenges helps guide architectural improvements, sprint planning, and risk mitigation strategies.

11.1. Key Risks

- **Cross-context data consistency:** Reliance on asynchronous event processing may lead to eventual consistency delays or data drift if events fail or are misordered.
- **Supabase dependency:** Heavy reliance on Supabase for auth, storage, and real-time sync limits flexibility and may pose migration or cost risks.
- **WebSocket and CDC coupling:** Real-time updates tied to Supabase's change data capture may limit observability and introduce debugging complexity.
- **Jitsi integration:** Self-hosted video infrastructure may pose reliability or scaling challenges under high concurrency.

11.2. Technical Debts

- **Incomplete automated tests** in some bounded contexts, especially around event processing and failure scenarios.
- **Manually triggered backfills** for event-driven sync between Therapist and Client sessions.
- **Manual database rollout:** Database schema changes are currently applied manually, increasing the risk of errors and inconsistencies across environments.

12. Glossary

Term	Definition
------	------------

Bounded Context	A logical boundary within the system where a specific domain model is defined and maintained independently.
CDC (Change Data Capture)	Mechanism to detect and respond to changes in the database.
Design System	A shared system of UI standards, components, and tokens used to ensure consistent frontend design.
Event Outbox	A pattern for reliably domain events by first storing them in a database table.